

LUỒNG GIAO TIẾP MQTT CỦA THIẾT BỊ BƠM NHIÊN LIỆU

Tóm tắt theo mã nguồn firmware hiện tại

Ngày rà soát: 26/09/2026

Phiên bản tài liệu: Dự thảo 1 - dùng để đối chiếu và bổ sung yêu cầu

1. Mục đích và kết luận nhanh

Tài liệu này mô tả đúng hành vi đang có trong mã nguồn: thiết bị kết nối MQTT, nhận lệnh, phản hồi ACK, thao tác phím ảo, theo dõi quá trình bơm, đọc hóa đơn RS232 và gửi kết quả lên server.

Luồng điều khiển chính hiện đã hỗ trợ năm lệnh: `set_qr_money`, `set_qr_litter`, `set_price`, `enable_virtual_key` và `cancel_qr_money`. Ngoài ra firmware vẫn đăng ký năm topic cũ dưới `/station/...` để tương thích ngược.

Điểm quan trọng: ACK `result=ok` hiện chỉ xác nhận dữ liệu hợp lệ và tác vụ đã được tạo. ACK này chưa chứng minh bàn phím ngoài đã nhận đúng phím, bơm đã chạy hoặc giao dịch đã hoàn tất.

2. Cấu hình MQTT và định danh thiết bị

Hạng mục	Giá trị/hành vi hiện tại
Broker	mqtt://161.248.146.170:1883
Phiên bản giao thức	MQTT v5
QoS topic mới	Subscribe command QoS 1; publish ACK/telemetry/event QoS 1
Retain	false
Keep-alive MQTT	60 giây
Clean session	Tắt (disable_clean_session=true)
Tự kết nối lại	Bật
Client ID	Đọc mqttClientId từ NVS; fallback node_qr_001
Gateway ID	Đọc gw_pay từ NVS; fallback gw_pay_001
Device ID	Đọc deviceId từ NVS; fallback node_pay_001
Username	Không cấu hình trong đoạn code hiện tại
Password	Đang khai báo cứng trong firmware

Giá trị xuất xưởng hiện thấy trong nvs.csv: gw_pay_001, node_pay_004, MQTT client ID node_pay_004, firmware version 15, operation mode 1 và price 10000.

3. Danh mục topic chính

Các topic mới được ghép động từ NVS theo mẫu tbmq/payment/<gw_pay>/<deviceId>/...

Chiều	Topic	QoS	Mục đích
Server → thiết bị	tbmq/payment/<gw>/<node>/command	1	Nhận lệnh điều khiển
Thiết bị → server	tbmq/payment/<gw>/<node>/ack	1	Xác nhận đã chấp nhận hoặc từ chối lệnh
Thiết bị → server	tbmq/payment/<gw>/<node>/event	1	Báo giao dịch bơm kết thúc
Thiết bị → server	tbmq/payment/<gw>/<node>/telemetry	1	Heartbeat và dữ liệu trạng thái/đo lường

Ví dụ với dữ liệu NVS hiện tại:

```
tbmq/payment/gw_pay_001/node_pay_004/command
tbmq/payment/gw_pay_001/node_pay_004/ack
tbmq/payment/gw_pay_001/node_pay_004/event
tbmq/payment/gw_pay_001/node_pay_004/telemetry
```

4. Quy tắc chung khi nhận command

- Thiết bị chỉ xử lý giao thức mới khi topic khớp chính xác topic command đã dựng từ NVS.
- Payload phải là JSON hợp lệ.
- cmd phải là chuỗi; param phải là object.
- msg_id là bắt buộc và phải đúng 32 ký tự hexadecimal. Code cũng chấp nhận alias cmd_id, command_id hoặc ack_to_id, nhưng phản hồi luôn dùng trường msg_id.
- Giá trị số có thể được gửi dưới dạng JSON number hoặc chuỗi chỉ chứa chữ số.
- Nếu hợp lệ, thiết bị tạo tác vụ tương ứng và gửi ACK. Nếu lỗi, thiết bị gửi ACK result=error kèm description.

ACK thành công chung:

```
{
  "ts": 1730970001,
  "ack_to": "set_qr_money",
  "result": "ok",
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011"
}
```

ACK lỗi chung:

```
{
  "ts": 1730970001,
  "ack_to": "set_qr_money",
  "result": "error",
  "description": "invalid qr_money",
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011"
}
```

Nếu chính msg_id bị thiếu hoặc sai định dạng, ACK lỗi không thể lặp lại msg_id; description là missing_or_invalid_msg_id.

5. Lệnh set_qr_money

5.1 Request

```
{
  "cmd": "set_qr_money",
  "param": { "qr_money": 50000 },
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30014"
}
```

5.2 Hành vi

1. Kiểm tra qr_money là số không âm trong phạm vi uint32_t.
 2. Với MAIN_RS232=1, giữ nguyên số tiền để nhập bằng phím; với MAIN_RS232=0, giá trị nhập được chia 100.
 3. Tạo payment context chứa loại MONEY, số tiền yêu cầu và msg_id.
 4. Tạo tác vụ phím ảo: nhấn \$, đợi 700 ms, nhập từng digit, rồi nhấn E.
 5. Gửi ACK ok ngay sau khi tạo tác vụ thành công.
 6. Khi chuỗi phím kết thúc, cho phép CONTROL_SWITCH GPIO7 bấm theo INPUT_SWITCH GPIO6.
- Lỗi có thể trả về: invalid qr_money, out of memory, task create failed.

6. Lệnh set_qr_litter

Tên command hiện được giữ đúng theo code là set_qr_litter (hai chữ t), không phải set_qr_liter.

6.1 Request

```
{
  "cmd": "set_qr_litter",
  "param": { "qr_litter": 23 },
  "msg_id": "f658159b2bda48499a605d21261de0ac"
}
```

6.2 Hành vi

1. Kiểm tra qr_litter hợp lệ và không vượt giới hạn khi nhân 10.
2. Tính giá trị nhập bàn phím là qr_litter × 10 để bù một chữ số thập phân. Ví dụ 23 lít được nhập thành 230.
3. Tạo payment context loại LITTER và lưu msg_id.
4. Tạo tác vụ phím ảo: nhấn C, nhấn L, đợi 700 ms, nhập từng digit, rồi nhấn E.
5. Gửi ACK ok sau khi tạo tác vụ thành công.

Mọi lỗi cấu hình lít hiện trả description=fail_config_litter.

7. Lệnh set_price

7.1 Request

```
{
  "cmd": "set_price",
  "param": { "price": 25000 },
  "msg_id": "211690e3252046f5a4f120accb8b4da9"
}
```

7.2 Hành vi

price phải nằm trong 0...65535. Mỗi lần nhận lệnh, firmware vẫn thực hiện chuỗi phím kể cả giá mới bằng giá cũ. Sau khi tạo tác vụ, giá mới được ghi vào NVS ngay và ACK ok được gửi.

Cấu hình	Chuỗi phím hiện tại
MAIN_RS232=1	C C P 1 2 3 4 5 6 E → các digit giá → E
MAIN_RS232=0	C C T P 0 1 2 E → 2 2 2 2 2 E → các digit giá → E

Lưu ý: giá được lưu flash trước khi có xác nhận từ thiết bị ngoài rằng chuỗi phím đã được nhận thành công.

8. Lệnh enable_virtual_key

```
{
  "cmd": "enable_virtual_key",
  "param": { "enable_virtual_key": 1 },
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011"
}
```

Giá trị	Hành vi
1	Bật phím ảo ESP32, chọn route virtual và bật master scan
0	Tắt phím ảo, chọn bàn phím vật lý ngoài và tắt master scan

Trạng thái route được lưu NVS nên vẫn được khôi phục sau reset/OTA. Giá trị khác 0 hoặc 1 trả lỗi invalid enable_virtual_key.

9. Lệnh cancel_qr_money

```
{
  "cmd": "cancel_qr_money",
  "param": { "cancel_qr_money": 1 },
  "msg_id": "aa4a23d06e2d4bd4a9a98272c3b1e841"
}
```

Thiết bị tạo tác vụ nhấn phím C, cố gắng xóa payment context có msg_id phù hợp và gửi ACK. Giá trị khác 1 trả lỗi invalid cancel_qr_money.

Điểm cần lưu ý: nếu msg_id của lệnh cancel không khớp context đang chạy, hàm xóa không xóa giao dịch đó, nhưng code hiện vẫn gửi ACK ok sau khi tác vụ phím được tạo.

10. State machine giao dịch bơm

Luồng set_qr_money và set_qr_litter dùng chung payment context và cùng cơ chế kết thúc:

1. Server gửi command có msg_id.
2. Thiết bị kiểm tra JSON, tạo tác vụ phím ảo, lưu context và gửi ACK.
3. Tác vụ phím ảo kết thúc; nếu đang ở route phím ảo thì chờ "keypad done" cho phép GPIO7 bấm GPIO6.
4. GPIO6 chuyển 0 → 1: xác nhận bơm bắt đầu. GPIO7 được đưa lên 1 khi điều kiện cho phép. Bộ thu hóa đơn RS232 được reset tại thời điểm này.
5. GPIO6 chuyển 1 → 0: xác nhận bơm kết thúc. GPIO7 về 0.
6. Firmware chờ tối đa 2000 ms, kiểm tra mỗi 50 ms để lấy hóa đơn RS232 hoàn chỉnh.
7. Có hóa đơn: gửi event completed với số tiền, số lít và đơn giá thực tế.

8. Không có hóa đơn: vẫn gửi event completed nhưng thêm trạng thái lỗi, đồng thời ép money=0 và liter=0.
9. Sau lệnh publish, payment context được xóa.

Điều kiện chống kết thúc giả đã có: mức GPIO6 bằng 0 ban đầu không được xem là hoàn tất. Firmware bắt buộc phải thấy trạng thái lên 1 trước, sau đó mới chấp nhận cạnh xuống 0.

11. Thu thập kết quả từ RS232

UART RX nhận dữ liệu hóa đơn dạng ASCII và bộ collector ghép dữ liệu qua nhiều chunk. Parser tìm ba nhãn:

Nhãn hóa đơn	Trường MQTT	Xử lý
Thanh tiền	money	Loại dấu phân cách hàng nghìn, chuyển thành số nguyên
Số lít	liter	Chuyển về milliliter nội bộ, xuất JSON với ba chữ số thập phân
Đơn giá	price	Loại dấu phân cách và chuyển thành số nguyên

Ví dụ dữ liệu RS232:

```
Thanh tien:      26.970(vnd)
Số lít:          0,93(lit)
Đơn giá:         29.000(d/l)
```

Kết quả tương ứng: money=26970, liter=0.930, price=29000.

12. Event giao dịch hoàn tất

12.1 Có hóa đơn RS232 hợp lệ

Topic: tbmq/payment/<gw>/<node>/event

```
{
  "ts": 1736900030,
  "event": "completed",
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011",
  "money": 26970,
  "liter": 0.930,
  "price": 29000
}
```

Các giá trị là dữ liệu thực tế đọc từ RS232, không lấy lại giá trị yêu cầu ban đầu.

12.2 Không nhận đủ hóa đơn trong 2000 ms

```
{
  "ts": 1736900030,
  "event": "completed",
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011",
  "result": "error",
  "description": "missing_rs232_receipt",
  "money": 0,
  "liter": 0,
  "price": 29000
}
```

Luồng lỗi này áp dụng chung cho cả set_qr_money và set_qr_litter. price vẫn là giá hiện tại trong NVS/RAM; chỉ money và liter bị ép về 0.

13. Telemetry định kỳ

Task ping_tb gửi heartbeat mỗi 60 giây lên topic telemetry.

```
{
  "ts": 1736900000,
  "msg_id": "9f1c2ab34d5e4f67a8b9c0d1e2f30011",
  "DevID": "node_pay_004",
  "fuel_type": "diesel",
  "keep_alive": 1,
  "RSSI": -52,
  "enable_virtual_key": 1,
  "version": 15,
  "price": 29000
}
```

msg_id chỉ xuất hiện khi đang có payment context hoạt động. enable_virtual_key=1 nghĩa là đang dùng phím ảo; bằng 0 nghĩa là route bàn phím vật lý.

Ngoài heartbeat, code còn một task telemetry cũ đọc uplink_queue và có thể gửi liter, money, price, RSSI cùng object data. Tuy nhiên khi RS232_RX_RAW_MONITOR=1, parser ATC cũ bị bỏ qua; vì vậy cần kiểm thử lại việc task cũ có còn nhận đủ dữ liệu để publish hay không.

14. Các topic tương thích cũ vẫn đang đăng ký

Topic legacy	QoS	Hành vi hiện tại
/station/End_Session/<deviceId>	0	Thực hiện chuỗi phím kết thúc phiên
/station/price/<deviceId>	0	Đổi giá bằng luồng cũ, không dùng ACK chuẩn mới
/station/fw_version/<deviceId>	0	Nếu version lớn hơn thì lưu NVS, đổi operation mode và restart
/station/deviceID/<deviceId>	0	Ghi device ID mới vào NVS rồi restart
/station/qr_price/<deviceId>	0	Nhập tiền bằng phím ảo nhưng không tạo payment context chuẩn mới

Topic legacy qr_price có thể nhận các tên trường qr_price, amount, money hoặc price. Do đường này không dùng đầy đủ msg_id, ACK và state machine mới, server nên ưu tiên topic tbmq/payment/.../command.

Một nhánh legacy khác gửi payload Client_End_Session vào chính topic ACK mới. Đây không phải định dạng ACK chuẩn và nên được xác nhận hoặc tách riêng.

15. Khởi động, reconnect và OTA

Khi khởi tạo, firmware dựng topic từ NVS, khởi động MQTT, cấu hình RS232, thực hiện boot clear bàn phím ảo, tạo task gửi dữ liệu và heartbeat, rồi khởi động bộ kiểm tra OTA GitHub.

Khi MQTT kết nối, thiết bị subscribe topic command mới với QoS 1 và năm topic legacy với QoS 0. Khi mất kết nối, cờ mqtt_connected bị xóa; thư viện được cấu hình tự reconnect.

Nếu chưa kết nối, hàm publish chỉ ghi log "MQTT not connected, skip topic=..." và trả lỗi; hiện chưa có hàng đợi ứng dụng để giữ lại ACK/event và gửi lại sau.

OTA GitHub hiện là luồng mạng riêng, không phải MQTT chính. Bộ kiểm tra chạy mỗi 30 giây, đọc `version.json`; khi build number remote lớn hơn version NVS thì tải firmware HTTPS và reboot. Topic legacy `/station/fw_version/...` vẫn tồn tại như một đường OTA cũ.

16. Những điểm cần xác nhận trước khi chốt protocol

1. [] ts phải là Unix epoch thật. Log thực tế từng có giá trị rất nhỏ như 590, cho thấy thời gian có thể chưa được đồng bộ SNTP.
2. [] Quy ước ACK ok là "đã nhận/tạo task" hay phải đợi thiết bị ngoài xác nhận chuỗi phím thành công.
3. [] Khi mất RS232, có nên dùng `event=completed` cùng `result=error`, hay đổi thành `event failed`.
4. [] Khi publish completion event thất bại do mất MQTT, code hiện vẫn xóa `payment context`; có cần lưu flash/hàng đợi để gửi lại không.
5. [] Có cần chống command trùng `msg_id` (idempotency) và từ chối command mới khi đang có giao dịch hoạt động không.
6. [] `cancel_qr_money` có cần trả lỗi nếu `msg_id` không khớp giao dịch đang chạy không.
7. [] Có tiếp tục duy trì các topic `/station/...` hay đặt thời hạn loại bỏ để tránh hai đường điều khiển song song.
8. [] Có chuyển broker sang `mqtts://`, bỏ password khai báo cứng và dùng credential riêng cho từng thiết bị không.
9. [] Giá trong NVS có nên chỉ cập nhật sau khi thiết bị ngoài xác nhận đổi giá thành công không.
10. [] Cần thống nhất chính tả API litter hay chuyển phiên bản mới sang liter mà vẫn duy trì tương thích.

17. Ma trận phản hồi nhanh

Command	ACK ngay	Tạo payment context	Chờ GPIO6 1→0	Đọc RS232	Gửi event
set_qr_money	Có	Có	Có	Có	Có
set_qr_litter	Có	Có	Có	Có	Có
set_price	Có	Không	Không	Không	Không
enable_virtual_key	Có	Không	Không	Không	Không
cancel_qr_money	Có	Xóa nếu khớp	Không	Không	Không

18. Nguồn mã đã rà soát

- main/wifi_mqtt_FD.c: kết nối MQTT, topic, command, ACK, telemetry và completion event.
- main/virtual_keypad.c: chuỗi phím ảo và timing thao tác.
- main/rs232.c: GPIO6/GPIO7, UART RX, collector và parser hóa đơn.
- main/ota_update.c: kiểm tra version và OTA GitHub.
- main/main.cpp và nvs.csv: nạp cấu hình NVS, ID, version và giá mặc định.

19. Ý kiến bổ sung của dự án

1.
2.

3.
4.
5.